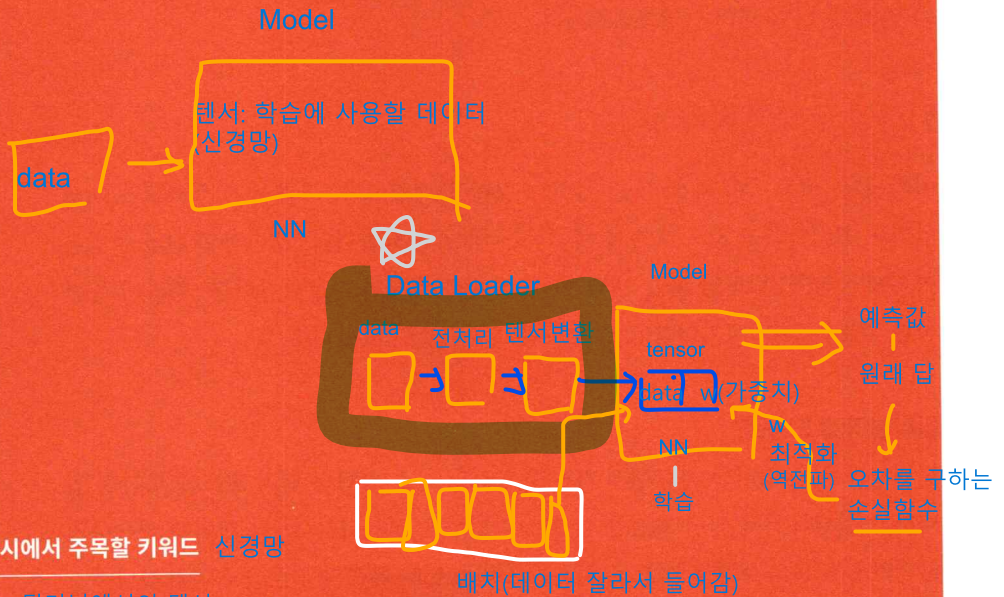


2일 차

딥러닝 기초

pytorch : 딥러닝(신경망) 라이브러리



이번 차시에서 주목할 키워드 신경망

- 텐서 딥러닝에서의 텐서
- 배치
- 데이터 로더 (적재)
- 가중치 (weight)
- 손실 함수 cost function
- 최적화 optimizer

2일 차 딥러닝 기초

The diagram illustrates the structure of deep learning code, divided into two main sections: ML (Machine Learning) and DL (Deep Learning).

ML Section:

- 데이터 전처리 (Data Preprocessing):** This section is divided into two parts:
 - 전처리1 (Preprocessing 1):** Includes **NaN 조치 (NaN Handling)** and **가변수화 (Feature Engineering)**.
 - 전처리2 (Preprocessing 2):** Includes **텐서로 변환 (Convert to Tensor)** and **Data, Loader (train set)**.
- 모델링 (Modeling):** This section includes **모델 설계 (Model Design)**, **학습 (Training)**, and **예측 및 검증 평가 (Prediction and Evaluation)**.

DL Section:

- 모델 설계 (Model Design):** This section includes **손실 함수 (Loss Function)** and **최적화 : w 학습 (Optimization : w Learning)**.
- 학습 (Training):** This section includes **학습 곡선 (Learning Curve)** and **학습 종료 후에는 w만 남음 (w는 파라미터가 됨) (After training ends, only w remains (w becomes the parameter))**.
- 예측 및 검증 평가 (Prediction and Evaluation):** This section includes **feed forward learning**.

Legend:

- ML:** Machine Learning
- DL:** Deep Learning
- 손실 함수 (Loss Function):** 손실 함수
- 최적화 : w 학습 (Optimization : w Learning):** 최적화 : w 학습
- 학습 곡선 (Learning Curve):** 학습 곡선
- 학습 종료 후에는 w만 남음 (w는 파라미터가 됨) (After training ends, only w remains (w becomes the parameter)):** 학습 종료 후에는 w만 남음 (w는 파라미터가 됨)
- feed forward learning:** feed forward learning

[그림 2-1] 딥러닝 코드 구조

The diagram illustrates a single layer of a neural network. A blue box represents the layer, containing three vertical rectangles representing neurons. A red arrow labeled 'FFL(내적)' (Forward Pass / Inner Product) points from the input (represented by a blue arrow labeled 'learning') into the layer. A red arrow labeled 'BWL' (Backward Pass / Weight Update) points from the output of the layer back to the input. A green arrow labeled 'layer' points from the output of the layer to the next layer (represented by a blue arrow labeled 'learning'). Below the diagram, the text 'FFL(내적)' is followed by a colon and the Korean text '유사도 찾기 (관계)' (Finding similarity (relationship)).

FFL(내적)
: 유사도 찾기
(관계)

55

딥러닝 코드 절차

앞에서 설명한 딥러닝 코드 구조를 절차대로 정리하면 다음과 같습니다.

- **환경 준비**: 딥러닝 학습에 필요한 라이브러리들을 불러오고, 딥러닝 학습에 사용할 디바이스(GPU 혹은 CPU)를 지정합니다.
- **데이터 전처리 1**: 데이터를 불러와서 전처리를 수행합니다. 여기에는 머신러닝과 동일한 전처리 과정(데이터 분할, 스케일링 등)이 포함됩니다.
- **데이터 전처리 2**: 딥러닝에서 필요한 전처리를 수행합니다. ~~데이터~~ ^{데이터} **텐서 변환**, **데이터 로더 생성** 등이 여기에 포함됩니다. 주의
- **모델 선언**: 딥러닝 모델 구조를 설계하고 옵티마이저와 손실 함수를 정의합니다.
- **학습**: 모델을 학습시킵니다. 이때 데이터를 반복해 학습시킵니다.
- **예측 및 검증**: 학습된 모델을 검증 평가합니다.

이 절차를 기억하면서 딥러닝 코드를 작성하겠습니다.

딥러닝 코드 작성하기

딥러닝 모델링을 간단히 살펴보기 위해 1일 차에서 사용했던 '광고비에 따른 매출액' 데이터를 사용합니다.

환경 준비

앞에서 만든 구글 드라이브 폴더에서 마우스 오른쪽 버튼을 클릭한 다음, [더 보기] - [Google Colaboratory] 메뉴를 차례로 선택해 새 코랩 파일을 생성합니다. 파일 이름은 '2일차_딥러닝실습.ipynb'로 변경합니다.

이어서 딥러닝 모델링을 위한 개발 환경을 준비합니다. 다음과 같이 코드를 입력하고 **Shift** + **Enter** 키를 눌러 실행합니다.

```
CODE
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import *
```

1일차에서 살펴보았던 기본 라이브러리들을 불러옵니다

다음으로 딥러닝에서 사용할 파이토치 라이브러리와 함수들도 불러옵니다.

CODE

```
import torch ①
from torch import nn ②
from torch.optim import Adam ③
from torch.utils.data import TensorDataset, DataLoader ④
```

- ① 파이토치 라이브러리를 불러옵니다. 파이토치는 딥러닝 모델을 만들고 학습할 때 필요한 핵심 도구입니다.
- ② nn은 신경망을 만들 때 자주 사용하는 모듈(Neural Network, nn)입니다.
- ③ 모델을 학습시킬 때 가중치(파라미터) 업데이트의 최적화 알고리즘 중 하나인 Adam을 불러옵니다.
- ④ 파이토치에서 데이터를 다룰 때 사용하는 도구들입니다.

필요한 라이브러리와 함수를 불러왔다면 다음으로 디바이스(Device)를 정합니다. 디바이스란 용어는 다양한 의미로 사용될 수 있는데, 여기서는 딥러닝 모델링 과정에서 수행되는 연산 처리 장치(CPU 혹은 GPU)를 지칭합니다.

CODE

```
device = "cuda" if torch.cuda.is_available() else "cpu" ①
print(device)
```

OUTPUT

cpu

- ① GPU(cuda)를 사용할 수 있으면 cuda를, 그렇지 않으면 cpu를 device 변수에 저장합니다.

머신러닝은 대부분 CPU만을 지원하므로 디바이스를 따로 설정하지 않습니다. 그러나 딥러닝은 연산 효율과 처리 속도를 위해서 GPU를 지원합니다. 따라서 이와 같이 현재 개발 환경에서 GPU를 사용할 수 있는지 확인하고 지정합니다. 코랩 파일을 생성하면 기본으로 설정되는 연산 장치는 cpu이므로 코드의 결과는 cpu로 출력됩니다. GPU 사용은 이번 차시의 '텐서와 데이터 로더 다루기' 절에서 좀 더 자세히 살펴보겠습니다.

데이터 전처리 1: 머신러닝과 공통

이 단계는 머신러닝과 딥러닝 모두 동일합니다. 데이터를 정제하고 정리해서 모델이 학습하기 좋은 상태로 만드는 기본 과정입니다.

‘광고비에 따른 매출액’ 데이터 불러오기, x, y 분할, 학습 및 검증 데이터 분할, 그리고 스케일링을 수행하는 전처리 코드를 다음과 같이 작성합니다.

CODE

```
# 데이터 불러오기
path = 'https://bit.ly/advertising_csv'
adv = pd.read_csv(path)

# x, y 분할
target = 'Sales'
x = adv.drop(target, axis=1)
y = adv.loc[:, target]

# 학습/검증 데이터 분리
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=0.2,
                                                    random_state=20)
print(type(x_train), type(x_val)) ①

# 스케일링
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)
print(type(x_train), type(x_val)) ②
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'> <class 'pandas.core.frame.DataFrame'> ①
<class 'numpy.ndarray'> <class 'numpy.ndarray'> ②
```

코드를 모두 입력하고 실행했나요? 여기서 한 가지 살펴볼 것이 있습니다. 스케일링 전의 x_train과 x_val은 판다스의 데이터프레임(①)인데, 스케일링 후에 저장된 x_train과 x_val은 넘파이 배열(②)이라는 점입니다. 머신러닝에서는 데이터 세트가 넘파이 배열이든 판다스 데이터프레임이든 상관없지만, 딥러닝에서는 텐서 변환을 위해 데이터 세트가 반드시 넘파이 배열이어야 합니다.

데이터 전처리 2: 딥러닝에 필요한 전처리

딥러닝에서는 기본적인 전처리 외에도 추가적인 준비 과정이 필요합니다. 바로 데이터를 텐서(Tensor)로 변환하고 데이터 로더(DataLoader)를 활용해 모델이 효율

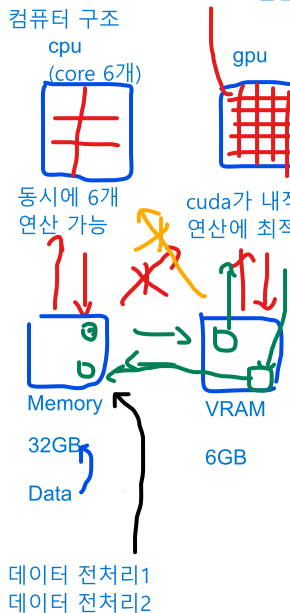
적으로 학습할 수 있도록 구성하는 과정입니다. 파이토치로 딥러닝 모델을 구현하려면 모든 데이터가 반드시 텐서 형태여야 하며, 학습 과정을 체계적이고 효율적으로 진행하기 위해 데이터 로더를 사용해야 합니다.

텐서와 데이터 로더에 대한 구체적인 설명과 코드는 이번 차시의 세 번째 절에서 다룹니다. 여기서는 우선 개념만 간단히 살펴보고 넘어갑니다.

- 텐서(Tensor) 변환: 딥러닝 프레임워크인 파이토치(PyTorch)나 텐서플로(TensorFlow)에서는 데이터를 일반 배열이 아닌 텐서라는 형식으로 변환해 사용합니다. 텐서는 다차원 배열로서 GPU 연산에 최적화된 자료구조입니다. 일반적으로 딥러닝에서는 파이토치의 `torch.tensor` 함수를 이용해 넘파이 배열을 텐서로 변환합니다.
- 데이터 로더(DataLoader) 구성: 데이터 로더는 텐서로 변환된 데이터를 미니배치(Mini Batch) 단위로 묶어 모델에 공급하는 역할을 합니다. 미니배치 단위로 데이터를 공급하는 방식은 GPU 메모리를 효율적으로 사용하고 학습을 안정적으로 진행하기 위해 필수적입니다. 예를 들어 1만 개의 데이터가 있다면 한 번에 모두 입력하는 대신, 32개씩 묶어 모델에 순차적으로 제공합니다. 미니배치 단위로 데이터를 제공하면 학습이 훨씬 안정적으로 진행되고 GPU 자원을 효율적으로 활용할 수 있습니다.

이제 텐서 변환 및 데이터 로더 구성을 위한 코드를 작성합니다.

1408개 병렬 연산
1660 기준
1408개
cuda : 연산



```
CODE
# 텐서 변환
x_train_ts = torch.tensor(x_train, dtype=torch.float32) ①
y_train_ts = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1) ②
x_val_ts = torch.tensor(x_val, dtype=torch.float32)
y_val_ts = torch.tensor(y_val.values, dtype=torch.float32).view(-1, 1)

# 디바이스 할당
x_train_ts, y_train_ts = x_train_ts.to(device), y_train_ts.to(device) ③
x_val_ts, y_val_ts = x_val_ts.to(device), y_val_ts.to(device)
```

- ① 넘파이 배열인 `x_train`을 파이토치 텐서로 바꾸고 데이터 타입을 계산에 적합한 32비트 실수형으로 지정합니다.
- ② 판다스의 시리즈 형태인 `y_train`에 `values` 속성을 적용해 넘파이 배열로 바꾼 다음, 파이토치 텐서로 변환합니다. 이때 `view(-1, 1)` 메서드로 1차원 배열을 2차원 형태(행 벡터→열 벡터)로 변경합니다. 이 내용 역시 '텐서와 데이터 로더 다루기' 절에서 좀 더 자세히 다룹니다. 검증 데이터(`x_val`, `y_val`)도 마찬가지로 변경합니다.

RAM -> VRAM으로
옮겨줘야 신경망 학습 가능

- ③ 학습용 텐서의 디바이스(여기서는 cpu)를 지정합니다. 연산 장치를 지정한다고 생각하면 됩니다. 검증용 텐서 역시 디바이스를 지정합니다.

사용할 데이터를 모두 텐서로 변환하고 디바이스까지 지정했으니 이제는 데이터 로더를 준비하겠습니다. 데이터 로더를 만드는 함수 `DataLoader`를 사용할 때는 텐서를 묶은 텐서 데이터 세트를 입력해야 합니다.

먼저 앞에서 텐서로 변경했던 학습 데이터(`x_train_ts`, `y_train_ts`)를 텐서 데이터 세트로 만들고 이 세트로 데이터 로더를 구성합니다.

CODE

```
BATCH_SIZE = 32 ①
train_ds = TensorDataset(x_train_ts, y_train_ts) ②
train_loader = DataLoader(train_ds, batch_size = BATCH_SIZE, shuffle=True) ③
```

- ① 한 번에 학습할 수 있는 배치 크기(데이터 개수)를 32로 정합니다.
② 학습 데이터 텐서(`x_train_ts`, `y_train_ts`)를 하나로 묶어 텐서 데이터 세트로 만듭니다. 이 작업은 데이터 로더 구성을 위해 꼭 필요합니다.
③ `DataLoader` 함수에 학습용 텐서 데이터 세트를 넣고 배치 크기를 지정합니다. 또한 배치를 만들 때 데이터를 섞도록 설정(`shuffle=True`)합니다.

`DataLoader` 함수는 데이터를 미니배치 단위로 잘라서 섞고 디바이스로 전달하는 일을 수행합니다. `DataLoader` 함수가 이 작업을 원활하게 수행하려면 인덱스로 접근할 수 있고 `x`와 `y`를 짝지어 꺼낼 수 있는 데이터 구조여야 합니다. 그래서 `DataLoader` 함수는 `TensorDataset` 함수로 구성된 데이터 형식을 요구합니다. 어떤 함수에서 특정 형식의 데이터를 입력으로 요구한다 함은 함수 내부의 동작 방식이 이미 정해져 있기 때문입니다. 따라서 `DataLoader` 함수를 안정적으로 동작하게 하려면 미리 약속된 입력 형식을 지켜야 합니다.

모델링을 위한 모든 데이터 준비는 끝났습니다. 지금까지의 딥러닝 절차를 다시 한번 정리하면 다음과 같습니다.

- 환경 준비: 딥러닝 학습에 필요한 라이브러리 및 함수 로딩. 딥러닝 학습에 사용할 디바이스(GPU 혹은 CPU) 지정.
- 데이터 전처리 1: 데이터 로딩, `x`, `y` 분할, 학습 및 검증 데이터 분할, 스케일링
- 데이터 전처리 2: 데이터를 텐서로 변환, 학습 데이터를 텐서 데이터 세트로 변환, 데이터 로더의 구성

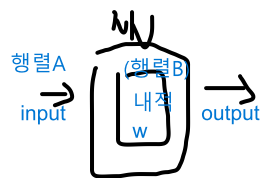
여러분은 이 흐름을 언제든지 머릿속에 그릴 수 있어야 합니다. 그럼 모델링으로 넘어갑시다.

딥러닝 모델링: 모델 선언

이번 단계는 딥러닝 모델을 직접 설계하고 학습시키며 평가하는 모델링 단계입니다. 딥러닝에서 모델링이란 모델이 데이터를 어떻게 받아들이고 어떤 구조로 연산하며 어떤 방식으로 학습하고 평가할지 정하는 과정입니다.

앞에서 소개했던 [그림 2-1]을 보면 모델링 부분은 모델 설계, 손실 함수와 옵티마이저 지정, 학습, 학습 곡선, 예측 및 평가로 이루어져 있습니다. 이 순서에 맞게 이번 모델링 단계도 전체 흐름을 파악하는 데 초점을 두면서 작성합니다.

먼저 모델을 선언하는 부분입니다. 모델 구조를 설계하는 부분이기도 합니다. 딥러닝 모델은 여러 개의 층(Layer)과 노드(Node)로 구성됩니다. 우선 다음과 같이 층이 하나인 간단한 구조의 딥러닝 모델을 작성합니다.



A: 3X2
B: 3X3
내적 불가. (2, 3)

CODE

열을 가져옴.

```
in_features = x_train_ts.shape[1] ①
model = nn.Sequential(             ②
    nn.Linear(in_features, 1),      ③
).to(device)                       ④
```

- ① shape 속성은 2차원 텐서의 행과 열의 길이(개수)를 출력합니다. [0]은 행의 길이, [1]은 열의 길이입니다. '광고비에 따른 매출액' 데이터에서 열의 길이는 입력변수(TV, Radio, Newspaper)의 개수이므로 3을 변수 in_features에 저장합니다.
- ② nn.Sequential은 딥러닝 모델을 선언하는 함수입니다. 이 함수에서 각각의 층(Layer)을 구성할 수 있습니다.
- ③ nn.Linear 함수로 한 개의 층을 구성합니다. 층에는 입력변수(정보)가 3개, 출력값은 1개가 있습니다..
- ④ 모델을 학습할 때 사용할 디바이스를 지정합니다. 이 모델의 디바이스는 cpu입니다.

이 딥러닝 모델은 3개의 변수(TV, Radio, Newspaper)를 이용해 매출액(Sales)을 예측합니다. nn.Linear(3, 1)은 입력 정보 3개로 예측값 1개를 출력하는 구조입니다.

손실 함수와 옵티마이저 구성

모델 구조를 선언했으니 이어서 손실 함수와 옵티마이저를 지정합니다. 모델을 학습시키기 위해서는 오차를 측정하는 손실 함수와 오차를 줄이기 위한 옵티마이저

의 설정이 필요합니다. 다음 코드를 작성하고 실행합니다.

CODE

```
loss_fn = nn.MSELoss() ①  
optimizer = Adam(model.parameters(), lr=0.01) ②
```

- ① 딥러닝의 회귀 모델에서는 손실 함수로 MSELoss(평균 제곱 오차 손실)를 사용합니다.
- ② 모델의 가중치를 업데이트할 최적화 알고리즘으로 Adam을 사용합니다. Adam에서 lr 옵션은 학습률을 의미합니다.

손실 함수와 옵티마이저를 간략히 살펴보겠습니다.

- 손실 함수: 예측값이 목표값과 얼마나 차이가 나는지 계산합니다. 개별 데이터에서 구한 오차의 전체 평균으로 계산합니다. 손실 함수를 목적 함수라고도 하는데, 이 손실 함수에서 계산된 값(흔히 오차라고 함)을 최소화하는 작업이 모델링 과정의 중요한 목적이기 때문입니다. 회귀 또는 분류 문제에 따라 각각 사용하는 손실 함수가 다릅니다. 이들 손실 함수는 3~5차시에서 모델링 유형에 맞게 자세히 설명합니다.
- 옵티마이저: 학습 과정에서 오차(손실)를 줄이는 방법은 모델의 가중치를 조절하는 일인데, 이 역할을 옵티마이저가 수행합니다. Adam은 옵티마이저의 한 종류로서 최근 딥러닝 모델링에서 가장 성능이 우수하다고 알려져 있습니다. 또한 옵티마이저가 가중치를 조절할 때 얼마만큼 조절할지를 결정하는 비율이 바로 학습률(lr, Learning Rate)입니다.

옵티마이저와 학습률은 딥러닝 학습의 방향과 속도를 결정하는 핵심 요소입니다. 이 두 요소는 이번 차시 마지막 절에서 다시 한번 다룹니다.

딥러닝 모델링: 학습

이제 모델을 학습시키겠습니다. 딥러닝에서 학습은 반복적이고 점진적인 방식으로 이루어지는데, 과정이 조금 복잡합니다. 따라서 학습 과정을 간단하게 먼저 설명하고 코드를 작성하겠습니다.

1. 학습 데이터의 반복 학습: 딥러닝은 학습 데이터를 한 번만 학습하는 것이 아니라 여러 번 반복해서 학습합니다. 우리가 교과서를 한 번 읽고 끝내는 게 아니라, 여러 번 반복해 공부하는 방법과 비슷합니다. 학습 데이터 전체를 한 차례

학습하는 과정을 1에포크(Epoch)라고 합니다. 에포크는 '주기'를 의미하는 단위입니다.

2. 에포크마다 배치 단위 학습: 1에포크마다 학습 데이터를 배치 단위로 학습시킵니다. 만약 전체 학습 데이터가 10건이고 배치 크기가 3이면 총 4개의 배치로 구성됩니다. 데이터 로더는 배치1(3건), 배치2(3건), 배치3(3건), 배치4(1건)와 같이 배치 단위로 총 4번에 걸쳐 데이터를 학습에 제공합니다.
3. 학습: 딥러닝에서 학습이란 현 데이터로 모델의 오차를 계산(오차의 순전파)하고 오차를 줄이기 위해 옵티마이저가 가중치를 조정(오차의 역전파)하는 과정입니다. 한 번 학습했다 함은 한 번 오차를 계산하고 가중치를 조정했다는 뜻입니다.
4. 검증: 에포크 단위로 학습된 모델의 성능을 검증합니다. 검증 데이터로 모델의 성능을 측정, 즉 오차를 계산합니다. 학습이 잘 진행되고 있다면 에포크가 거듭될수록 검증 오차는 줄어듭니다.



오차의 순전파, 역전파

딥러닝의 학습 과정은 크게 순전파와 역전파로 나눌 수 있습니다.

- 순전파(Forward Propagation): 입력 데이터를 모델에 넣어 연산을 거친 다음, 예측값을 계산하는 과정입니다. 이때 목표값과 비교해 오차를 구합니다.
- 역전파(Backward Propagation): 계산된 오차를 바탕으로 가중치가 얼마나 잘못되었는지를 역으로 전파해 가중치를 조금씩 조정하는 과정입니다.

순전파는 '예측하기', 역전파는 '틀린 만큼 가중치를 조정하기'라고 이해하면 쉽습니다.

이제 학습 코드를 작성합니다. 코드가 좀 길고 복잡해 보일 수 있지만, 앞에서 설명한 4가지 과정을 그대로 구현한 예시입니다. 여러분도 코드를 따라 작성하면서 딥러닝에서 학습은 어떻게 이루어지는지 전체적인 흐름을 익히기 바랍니다. 코드의 내용 중 일부는 3일 차에서 자세히 다룰 예정이므로 모르는 내용이 나오더라도 지금은 넘어가도 괜찮습니다.

CODE

```
EPOCHS = 100
tr_loss_list, val_loss_list = [], []
```

```

# 1. 학습 데이터 반복 학습
for epoch in range(EPOCHS): ①
    model.train() # 모델 학습 모드
    total_loss = 0.0
    n_batch = 0

    # 2. 에포크마다 배치 단위 학습
    for x, y in train_loader: ②
        x, y = x.to(device), y.to(device) ③

        # 3. 학습
        # 오차 순전파: 학습 데이터에서 오차 계산 ④
        pred = model(x) # 모델에 x를 입력해 예측
        loss = loss_fn(pred, y) # 오차 계산

        # 오차 역전파: 오차로부터 가중치 업데이트 ⑤
        optimizer.zero_grad() # 기울기(gradient) 초기화
        loss.backward() # 손실 함수를 기준으로 각 가중치의 기울기를 계산
        optimizer.step() # 기울기를 기준으로 가중치 조정

        total_loss += loss.item() ⑥
        n_batch += 1

    # 에포크마다 학습 오차 집계
    tr_loss = total_loss / n_batch ⑦

    # 4. 검증
    model.eval() # 모델 평가/추론 모드
    with torch.no_grad(): ⑧
        val_pred = model(x_val_ts)
        val_loss = loss_fn(val_pred, y_val_ts)

    tr_loss_list.append(tr_loss) ⑨
    val_loss_list.append(val_loss)
    print(f"Epoch {epoch+1 : 4d} | train loss {tr_loss:.4f} | val loss {val_loss:.4f}")

```

OUTPUT

```

Epoch 1 | train loss 178.0856 | val loss 172.9225
Epoch 2 | train loss 150.3507 | val loss 146.1269
Epoch 3 | train loss 124.8844 | val loss 122.5367
(중략)
Epoch 98 | train loss 3.3977 | val loss 5.5265
Epoch 99 | train loss 3.3786 | val loss 5.4755
Epoch 100 | train loss 3.3517 | val loss 5.4273

```

- ① 학습 데이터로 모델을 반복 학습시키는 for loop 구문입니다. 여기서는 학습을 100번 반복합니다.
- ② 한 번 학습할 때마다 배치 단위로 데이터를 추출해 모델에 제공하는 for loop 구문입니다.
- ③ 배치 x와 y를 처리할 디바이스를 지정합니다.

- ④ 오차를 순전파합니다. 순전파는 데이터를 모델에 넣고 예측한 다음, 목표값과 비교해 오차를 계산하는 과정입니다.
- ⑤ 오차를 역전파합니다. 순전파로 계산된 오차를 줄이기 위해 가중치를 조정하는 과정입니다. 옵티마이저가 이 역할을 맡아 처리합니다.
- ⑥ 이전 배치까지 누적 계산된 오차에 현재 배치의 오차를 더합니다. items 메서드는 이번 차시 '텐서와 데이터 로더 다루기' 절에서 배웁니다.
- ⑦ 하나의 에포크 과정이 끝나면 ⑥에서 합친 오차를 배치 수로 나누어 평균 오차를 계산합니다.
- ⑧ 검증 단계입니다. 한 에포크의 학습이 끝나면 검증 데이터를 이용해서 검증 성능(검증 오차)을 계산합니다. 이때 with torch.no_grad()는 다음 코드 블록에서는 기울기를 계산하지 않겠다는 의미입니다.
- ⑨ 한 에포크에서 계산된 학습 오차(tr_loss)와 검증 오차(val_loss)를 각각 리스트에 추가합니다. 학습이 모두 종료되면 학습 곡선을 그리기 위함입니다.

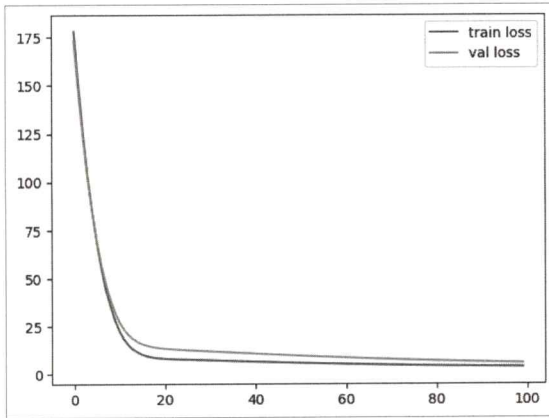
코드를 실행하면 에포크마다 학습 오차와 검증 오차가 출력 셀에 표시됩니다. 에포크가 진행될수록 두 오차가 조금씩 줄어드는 모습을 볼 수 있습니다. 다만 딥러닝 모델링 과정에서 무작위(Random) 요소가 포함되어 있기 때문에 책에 제시된 결과와 여러분이 직접 실행한 결과는 조금 다를 수 있습니다. 딥러닝에서는 보통 이런 무작위 요소를 받아들이고 모델링합니다.

딥러닝 모델링: 학습 곡선

딥러닝에서 학습이 적절하게 이루어졌는지 확인하기 위해 학습 곡선(Learning Curve)을 그립니다. 학습 곡선은 에포크마다 손실 함수값(오차)의 변화를 그래프로 그린 것으로 모델이 제대로 학습했는지 모니터링할 수 있습니다. 또한 학습 곡선은 모델의 학습 상태가 과적합(Overfitting)인지 아니면 과소적합(Underfitting)인지 판단할 수 있는 중요한 정보를 제공합니다.

저장한 학습 오차와 검증 오차 리스트(tr_loss_list, val_loss_list)로 다음과 같이 학습 곡선을 그립니다.

```
CODE
plt.plot(tr_loss_list, label='train loss') ①
plt.plot(val_loss_list, label='val loss')
plt.legend() ②
plt.show() ③
```



- ① matplotlib의 plot 메서드로 에포크마다 계산된 학습 오차와 검증 오차를 선 그래프로 시각화합니다.
- ② matplotlib의 plot 메서드에서 지정한 그래프의 레이블을 범례 형태로 표시합니다.
- ③ 그래프를 화면에 출력합니다.

그래프를 보는 기본 방법은 축의 의미가 무엇인지 확인한 다음, 그래프의 패턴이나 경향을 살펴보면 됩니다. 학습 곡선에서 x축은 에포크 수이고, y축은 손실 합숫값, 즉 오차입니다. 그래프를 보면 에포크가 증가할수록 오차가 급격히 줄어들다가 점차 기울기가 완만해지고 있습니다.

이 책의 그래프와 독자의 그래프 사이에 차이가 많이 나도 걱정할 필요는 없습니다. 데이터가 적을수록 학습 결과는 들쭉날쭉합니다. 다음 차시부터 다루는 데이터는 지금 예제 데이터보다 크기가 크므로, 책의 그래프와 유사한 학습 곡선이 그려질 겁니다. 여기서는 학습 곡선을 어떻게 그리는지 정도만 이해하면 됩니다. 학습 곡선을 해석하는 법 역시 3일 차에서 자세히 다룹니다.

딥러닝 모델링: 예측 및 검증 평가

학습이 끝난 다음에는 학습된 모델을 평가해야 합니다. 이를 위해 학습에서 사용하지 않은 검증 데이터나 테스트 데이터를 활용합니다. 검증 평가는 모델이 실전, 즉 운영 환경에서도 잘 작동할 수 있는지 판단하는 과정입니다.

학습된 모델로 검증 데이터를 예측한 다음, 회귀 모델 평가 지표인 RMSE, MAE, R^2 Score로 평가하겠습니다.

CODE

```
# 예측
model.eval() ①
with torch.no_grad(): ②
    pred_val = model(x_val_ts).numpy() ③

true_val = y_val_ts.numpy() ④

# 평가 ⑤
print(f"RMSE : {root_mean_squared_error(true_val, pred_val):.4f}")
print(f"MAE : {mean_absolute_error(true_val, pred_val):.4f}")
print(f"R2 : {r2_score(true_val, pred_val):.4f}")
```

OUTPUT

```
RMSE: 2.3297
MAE: 1.7562
R2: 0.8500
```

- ① 모델을 평가 모드로 전환합니다.
- ② with 문 이하 블록에서 torch.no_grad()로 기울기의 자동 계산(자동 미분)을 하지 않도록 합니다. 평가(예측)할 때는 기울기(Gradient, 그라디언트)를 계산할 필요가 없으므로 메모리와 속도를 절약합니다. 옵티마이저는 기울기를 계산해 가중치를 조정합니다. 옵티마이저는 이번 차시 마지막 절에서 다시 설명합니다.
- ③ 검증 데이터를 이용해서 예측합니다. 예측 결과는 텐서 데이터로 나옵니다. numpy 메서드로 예측 결과를 넘파이 배열로 변환하고 변수 pred_val에 저장합니다.
- ④ 검증 데이터의 정답에 해당하는 y_val_ts 텐서도 넘파이 배열로 변환합니다.
- ⑤ RMSE, MAE, R² Score로 평가하고 결과를 출력합니다. print 문의 {:.4f}는 평가 함수의 결과를 소수점 4자리로 출력(소수점 5자리에서 반올림)하라는 의미입니다.

학습 과정에서 무작위로 처리되는 부분이 있으므로 검증 평가 결과 역시 여러분과 필자의 결과는 다를 것입니다. 이 ‘무작위로 처리되는 부분’에 대해서는 이번 차시의 ‘딥러닝 학습 개념 이해’ 절에서 다시 설명합니다.

지금까지 딥러닝 모델링 순서에 맞춰 코드를 작성해 보았습니다. 전체 과정을 다시 한번 정리합니다. 이렇게 재차 정리하는 이유는 이 흐름을 어떤 경우에도 꼭 기억하라는 의미입니다.

- 환경 준비: 딥러닝 학습에 필요한 라이브러리 및 함수 로딩, 딥러닝 학습에 사용할 디바이스(GPU 혹은 CPU) 지정.
- 데이터 전처리 1: 데이터 로딩, x, y 분할, 학습 및 검증 데이터 분할, 스케일링
- 데이터 전처리 2: 데이터를 텐서로 변환, 학습 데이터를 텐서 데이터 세트로 변환, 데이터 로더 구성

- 모델 선언: 딥러닝 모델 구조를 설계하고 옵티마이저와 손실 함수를 정의
- 반복 학습: 학습 데이터를 n번 반복(n에포크)
- 한 에포크 내에서 배치 단위 학습: 데이터 로더로 배치 단위 학습
- 학습의 정의: 하나의 배치로 모델의 오차를 계산(오차의 순전파)하고 오차를 줄이기 위해 옵티마이저가 가중치를 조정(오차의 역전파)하는 과정
- 예측 및 검증: 검증 데이터로 예측하고 성능을 평가

지금까지 딥러닝 구현의 큰 흐름을 모두 짚어 보았습니다. 이어지는 절들에서는 딥러닝 모델링을 본격적으로 배우기 전에 꼭 알아야 할 중요한 개념인 텐서와 데이터 로더, 디바이스, 가중치를 좀 더 상세히 소개합니다. 본격적인 모델링을 배우기 전에 꼭 알아야 할 핵심 개념들을 미리 살피기 위한 과정입니다.

1. tensor() : 데이터 -> 텐서 변환
2. TensorDataset(): 학습데이터 + 레이블
3. DataLoader

딥러닝 학습 과정 이해 1: 텐서와 데이터 로더

이번 절에서는 딥러닝 모델링의 데이터 형식인 **텐서(Tensor)**의 기본 개념을 알아보고 간단한 예제들로 텐서를 직접 다루어 봅니다.

데이터타입(저장공간)
gpu 전용

머신러닝 -> ndarray 사용(cpu 전용)

텐서 이해하기

딥러닝 모델에서는 일반적인 데이터 구조(데이터프레임, 배열 등) 대신 **텐서(Tensor)**라는 특별한 데이터 구조를 사용합니다. 텐서는 다차원 배열로서 딥러닝 프레임워크(텐서플로, 파이토치 등)에서 데이터를 효율적으로 처리하기 위해 설계된 자료구조입니다. 딥러닝을 효과적으로 학습하고 활용함에 있어 텐서를 이해하는 일은 매우 중요합니다.

텐서: 딥러닝 처리하기 위한 자료구조

딥러닝 모델은 수많은 데이터를 학습하며 연산을 수행하는데, 이 과정에서 대규모 **행렬 연산**이 필수적으로 사용됩니다. 행렬 연산을 위해 **넘파이(NumPy)** 배열도 유용하지만, 특히 텐서는 GPU를 활용한 빠른 연산과 자동 미분(Auto Differentiation) 기능을 지원한다는 점에서 특별합니다.

딥러닝 전용

미분하면 기울기 나옴.
역전파 안에 미분이 들어감.

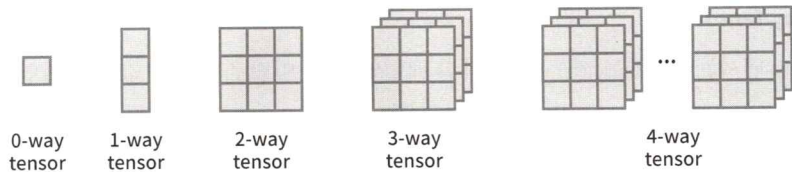
tensor = ndarray + @

[그림 2-2]는 차원(Dimension)에 따라 달라지는 텐서(Tensor)의 구조를 보여줍니다.

저장소.

텐서는 딥러닝에서 데이터를 표현하는 기본 단위로서 차원이 증가할수록 데이터의 복잡도는 커지지만, 표현할 수 있는 정보량은 많아집니다. [그림 2-2]를 바탕으로 각 차원의 텐서가 무엇을 의미하는지 코드로 살펴보겠습니다.

파이토치에서 데이터 저장하는 곳은 텐서 밖에 없음.



[그림 2-2] 차원에 따른 텐서 비교

새 코랩 파일을 만들고 파일 이름을 '2일차_텐서연습.ipynb'로 변경합니다. 첫 셀에서 기본적인 라이브러리를 몇 개 불러오겠습니다.

CODE

```
import torch
import numpy as np
import pandas as pd
```

텐서 실습에 꼭 필요한 라이브러리를 불러왔습니다. 이어서 텐서를 생성하고 다루는 방법을 살펴봅시다.

0-way Tensor

0-way 텐서는 스칼라(Scalar) 값 또는 단일 수치값을 뜻합니다. 예를 들면 정수 5나 실수 3.14 같은 단일 값이 바로 0-way 텐서입니다.

빈 코드 셀에서 다음과 같이 코드를 작성합니다.

CODE

```
scalar = torch.tensor(5) ①
print(scalar)
print(scalar.shape)      ②
```

OUTPUT

```
tensor(5)
torch.Size([])
```

- ① torch.tensor 함수에 단일 수치값 5를 입력해 0-way 텐서를 만듭니다.
- ② shape 혹은 Size 속성을 이용하면 텐서의 축(Axis)별 길이, 즉 축별 데이터의 개수를 표시할 수 있습니다. 텐서의 크기라고도 합니다.

코드를 실행하면 0-way 텐서는 tensor(5)로 표시됩니다. shape 속성의 결과는 torch.Size([])로서 값이 비어 있습니다. 단일 값은 축(차원)이 없기 때문입니다. 이처럼 축이 없는 형태의 텐서를 0-way 텐서라고 합니다.

1-way Tensor

1-way 텐서는 벡터(Vector) 혹은 1차원 배열입니다. 일렬로 나열된 값의 집합입니다.
다음 코드를 작성하고 실행합니다.

CODE

```
vector = torch.tensor([1, 2, 3, 4]) ①  
print(vector)  
print(vector.shape)
```

OUTPUT

```
tensor([1, 2, 3, 4])  
torch.Size([4])
```

| ① torch.tensor 함수에 1차원 리스트를 입력합니다.

코드를 실행하면 1차원 형태의 텐서인 `tensor([1, 2, 3, 4])`를 출력합니다. 1차원이므로 축은 1개이고 축의 길이(데이터 개수)는 4입니다. `shape` 속성을 이용해 출력하면 0-way 텐서는 축이 없어 비어 있었지만, 1-way 텐서는 길이가 4인 축이 하나 있습니다.

2-way Tensor

2-way 텐서는 행렬(Matrix) 형태의 2차원 구조입니다. 행(Row)과 열(Column)로 구성된 표 형식입니다.

CODE

```
matrix = torch.tensor([[1, 2, 3], [4, 5, 6]]) ①  
print(matrix)  
print(matrix.shape)
```

OUTPUT

```
tensor([[1, 2, 3],  
        [4, 5, 6]])  
torch.Size([2, 3])
```

| ① torch.tensor 함수에 2차원 리스트를 입력합니다.

코드를 실행하면 2차원 형태의 텐서인 `tensor([[1, 2, 3], [4, 5, 6]])`를 출력합니다. 2차원이므로 축은 2개입니다. 첫 번째 축의 길이는 2, 두 번째 축의 길이는 3, 즉 2행 3열입니다.

3-way Tensor

3-way 텐서는 3차원 배열입니다. 이는 2차원 배열(행렬)을 여러 개 쌓아 놓은 형태입니다. 대표적인 3차원 형태의 데이터가 컬러 이미지입니다. 컬러 이미지는 2차원 데이터와 3가지 색상(Red, Green, Blue)이 만나 이루어진 3차원 데이터 구조입니다.

3차원 텐서 코드를 다음과 같이 작성합니다.

CODE

```
tensor3d = torch.rand(2, 3, 4) ①
print(tensor3d)
print(tensor3d.shape)
```

OUTPUT

```
tensor([[[[0.5711, 0.4481, 0.3645, 0.2995],
          [0.8532, 0.1059, 0.2305, 0.1047],
          [0.7042, 0.6744, 0.9566, 0.0341]],

        [[0.5395, 0.8055, 0.8040, 0.9909],
          [0.7151, 0.4588, 0.3526, 0.9051],
          [0.9468, 0.0474, 0.5821, 0.7685]]]])
torch.Size([2, 3, 4])
```

① 3차원 텐서는 3차원 형태, 즉 축이 3개 있는 구성입니다. 3차원 리스트를 입력할 수도 있지만, 여기서는 `torch.rand` 함수로 무작위로 $2 \times 3 \times 4$ 의 행렬값을 채우는 3차원 텐서를 생성했습니다.

3차원 텐서의 결과가 제법 복잡해 보입니다. 또한 $2 \times 3 \times 4$ 크기의 3차원 텐서를 어떻게 읽어야 할지도 난감합니다. 이 3차원 텐서는 3×4 로 된 2차원 텐서가 2개 있다고 읽으면 됩니다. 3차원 텐서는 여러 개의 2차원 텐서로 이루어진 구조입니다.

그렇다면 4차원 텐서는 어떻게 읽어야 할까요? 일상에서는 보통 3차원 공간만을 상상할 수 있다 보니 4차원은 어렵게 느껴집니다. 4차원 텐서는 여러 개의 3차원 텐서가 모인 구조이고, 5차원 텐서는 여러 개의 4차원 텐서가 모인 구조라고 생각하면 조금 더 쉽게 받아들일 수 있습니다.

흑백 이미지 한 장은 보통 2차원으로 표현됩니다. 그런데 컬러 이미지는 색상(Red, Green, Blue)을 추가로 표현하므로 이미지 하나가 3차원(색상 $\times$ 세로 픽셀 수 $\times$ 가로 픽셀 수)입니다. 컬러 이미지(3차원)가 여러 개 있는 데이터 세트가 4차원 텐서 데이터라고 생각하면 조금 더 받아들이기 쉽습니다.

텐서 다루기

이제 텐서를 생성하고 조작하면서 딥러닝에서 데이터를 어떻게 다루는지 직접 익혀보겠습니다. 텐서를 다루는 방법은 처음부터 모두 외울 필요는 없습니다. 코드를 실행하고 여러 가지 조작을 경험하는 것만으로도 충분합니다. 이후 실제로 텐서 작업이 필요할 때 다시 찾아보고 적용하면 됩니다.

저장공간 종류, 크기 지정 종류 { int 8, 16, ...
float 32, 64, ...
... } VRAM의 효율적 공간 사용을 위해.

텐서의 데이터 타입 지정

인코딩 or 벡터화해아 텐서를 구성하는 값은 모두 숫자로서 이 숫자에는 타입을 지정할 수 있습니다. 기본적으로 정수형(int64)이나 실수형(float32) 타입을 많이 사용합니다. 딥러닝에서는 문제의 성격에 따라 특정 데이터 타입을 요구하는 경우가 있습니다. 예를 들어 분류 문제에서는 일반적으로 목표값(target)은 정수형(int64)으로, 연속적인 값인 입력값(feature)은 실수형(float32)으로 처리합니다.

다음 코드를 입력합니다.



CODE

```
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64) ① 생성하면서 타입 지정 가능
float_tensor = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float32) ②
print(int_tensor)
print(float_tensor)
```

OUTPUT

```
tensor([1, 2, 3])
tensor([1., 2., 3.])
```

- ① 텐서를 만들 때 dtype=torch.int64로 지정하면 값을 정수형으로 저장합니다. int64는 64비트 정수형을 의미하며 텐서에서 정수형을 표현할 때 사용하는 기본 타입입니다.
- ② dtype=torch.float32로 지정하면 값을 32비트 실수형 데이터로 저장합니다. 역시 텐서에서 실수형을 표현할 때 사용하는 기본 타입입니다.

실수형 텐서의 결과를 보면 각각의 값 뒤에 점(.)이 붙어 1., 2., 3.으로 표시됩니다. 소수점 이하에 값이 있으면 그대로 표현하지만, 1.0이나 2.0처럼 소수점 이하가 0이면 0을 생략하고 1., 2.과 같이 나타냅니다. 마찬가지로 0.1, 0.2는 앞의 0을 생략하고 .1, .2 형태로 표현합니다.

정수 및 실수 타입을 잘못 쓰면 학습 과정에서 오류가 발생하거나 성능이 떨어지므로 데이터 타입에 주의해야 합니다.



텐서 속성 확인

생성한 텐서에는 다양한 속성이 있습니다. 다음은 텐서를 다룰 때 자주 사용하는 속성들입니다.

CODE

```
print(float_tensor)
print(f"Shape: {float_tensor.shape}") ① 차원
print(f"Datatype: {float_tensor.dtype}") ① 데이터타입
print(f"Device: {float_tensor.device}") ② 저장된 장치(cpu, cuda)
```

OUTPUT

```
tensor([1., 2., 3.])
Shape: torch.Size([3])
Datatype: torch.float32
Device: cpu
```

- ① 텐서의 데이터 타입을 확인합니다.
- ② 텐서가 어느 디바이스에 할당되어 있는지 보여 줍니다.

실행 결과는 실수형 텐서의 결괏값, 1차원 텐서(float_tensor)의 길이, 1차원 텐서의 타입, 그리고 텐서를 처리하는 디바이스가 각각 무엇인지 보여 줍니다.

텐서 연산하기

딥러닝에서는 입력 데이터에 가중치를 곱하고 활성화 함수를 적용하고 손실을 계산하는 등 다양한 수치 연산들을 수행합니다.

먼저 간단한 사칙연산부터 시작하겠습니다. 다음 코드를 작성합니다.

CODE

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
print("덧셈:", a + b)
print("곱셈:", a * b)
print("제곱:", a ** 2)
```

1.0 + 4.0
2.0 + 5.0
3.0 + 6.0

원소별 연산

OUTPUT

```
덧셈: tensor([5., 7., 9.])
곱셈: tensor([ 4., 10., 18.])
제곱: tensor([1., 4., 9.])
```

파이토치에서 텐서 연산은 넘파이의 배열 연산과 비슷합니다. 기본적인 사칙 연산은 같은 위치의 값끼리(element-wise) 계산합니다. 예를 들어 두 텐서의 덧셈 연산

원소별 연산

의 결과를 보면 텐서 a의 첫 번째 값 1.과 텐서 b의 첫 번째 값 4.을 더해 결과 텐서의 첫 번째 값이 5.이 됩니다.

텐서 집계하기

텐서 연산에서 자주 사용하는 집계 ^{메소드}함수인 sum, mean, max를 알아봅니다. 다음과 같이 코드를 입력합니다.

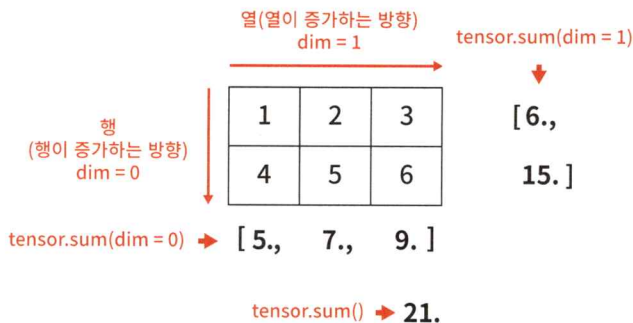
```
CODE
x = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32) ①
print("전체 합계:", x.sum()) ②
print("열별 합계:", x.sum(dim=0)) ③
print("행별 합계:", x.sum(dim=1)) ④
```

```
OUTPUT
전체 합계: tensor(21.)
열별 합계: tensor([5., 7., 9.])
행별 합계: tensor([ 6., 15.] )
```

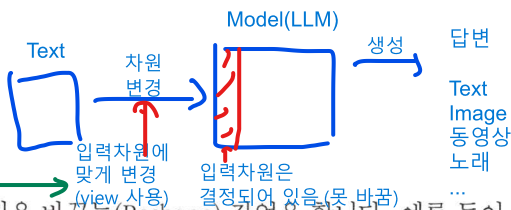
- ① 32비트 실수 타입으로 2차원 텐서를 선언합니다.
- ② 집계 함수 sum 메서드의 괄호에 아무것도 입력하지 않으면 데이터의 전체 합계를 계산합니다.
- ③ dim=0: 첫 번째 차원은 행입니다. 행 방향으로(즉, 열별로) 합계를 계산합니다.
- ④ dim=1: 두 번째 차원인 열 방향으로(즉, 행별로) 합계를 계산합니다.

dim=0일 때 행 방향으로 계산한다는 것이 이해가 되나요? 이해를 돕기 위해 [그림 2-3]으로 다시 설명합니다.

[그림 2-3]과 같이 2차원 텐서가 있을 때 dim=0은 행이 증가하는 방향으로 집계하라는 의미입니다. 따라서 sum(dim=0)의 결과는 [5., 7., 9.]입니다. 또한 dim=1은 열이 증가하는 방향으로 집계하라는 의미로 결과는 [6., 15.]입니다.



[그림 2-3] 텐서 집계 함수 이해하기



텐서 차원 변경

경우에 따라서 텐서의 차원을 바꾸는(Reshape) 작업을 합니다. 예를 들어 3행 2열의 2차원 텐서를 같은 차원의 2행 3열로 변경할 수 있고 차원을 늘리거나 줄일 수도 있습니다. 텐서의 차원을 변경할 때 사용하는 메서드로는 view 또는 reshape(넘파이나 판다스에서 사용하는 함수와 이름이 같음)가 있습니다.

코드로 연습하겠습니다.

CODE

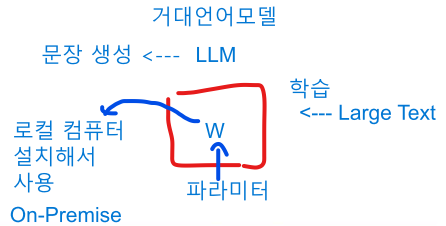
```
x = torch.tensor([[1, 2], [3, 4], [5, 6]])
print("원래 텐서:", x) ①
reshaped = x.view(2, 3) ② reshape 은 np쪽.
print("변형된 텐서:", reshaped)
```

1 2
3 4 → 1 2 3
5 6 4 5 6

OUTPUT

원래 텐서: tensor([[1, 2],
[3, 4],
[5, 6]])
변형된 텐서: tensor([[1, 2, 3],
[4, 5, 6]])

- ① 현재 텐서는 3행 2열입니다.
- ② 2행 3열로 변경합니다.



view 등의 메서드로 텐서 구조를 변경할 수 있습니다. **모델의 차원에 맞게 데이터를 입력해야 하는 경우 텐서 구조를 변경하는 작업들은 반드시 필요한 전처리 과정입니다.** 예를 들어 **이미지 한 장의 기본 구조는 3차원입니다.** 이미지를 데이터 세트로 만들면 4차원이 됩니다. 모델을 학습시키거나 학습된 모델로 예측할 때는 반드시 데이터를 4차원 구조 형태로 만들어 입력해야 합니다. 간혹 흑백 이미지를 2차원으로 저장하는 경우가 있습니다. 그때는 reshape 메서드 등을 사용해 2차원 이미지를 3차원으로 변환해야 합니다.

이미지: 평면(가로세로) + 색깔.
so 3차원

저장공간 텐서 연결과 분리

데이터를 다루다 보면 여러 **텐서를 하나로 합치거나 하나의 텐서를 여러 개의 텐서로 나누는 경우가 있습니다.**

먼저 두 개의 텐서를 **합치는 방법**을 설명합니다. 두 개의 텐서(2x2 텐서 a와 1x2 텐서 b)를 위아래(행 방향, dim=0)로 합치려면 **torch.cat** 함수를 사용합니다. cat은 Concatenate의 약자로 사슬로 엮는다는 의미입니다. 다음 코드를 입력하고 결과를 확인합니다.

CODE

```

a = torch.tensor([[1, 2], [3, 4]]) 2차원
b = torch.tensor([[5, 6]]) 2차원
concatenated = torch.cat([a, b], dim=0) ①
print("연결된 텐서:\n", concatenated)

```

차원이 같아야 cat 가능

a: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
b: $\begin{bmatrix} 5 & 6 \end{bmatrix}$ $\downarrow$ dim=0

OUTPUT

```

연결된 텐서:
tensor([[1, 2],
        [3, 4],
        [5, 6]])

```

- ① cat 메서드에서 리스트로 두 텐서를 묶어([a, b]) 입력하고 방향은 행(dim=0)으로 지정하면 두 텐서를 행 방향으로(위아래로) 합칩니다.

텐서를 cat으로 합칠 때 주의해야 할 점이 있습니다. 즉, 행 방향으로 합치려면 열 수가 같아야 하고 열 방향으로 합치려면 행 수가 같아야 합니다. 그래야 구조적으로 합칠 수 있습니다.

다음은 1차원 텐서 x를 크기가 3인 1차원 텐서 두 개로 나누는 split 메서드입니다.

CODE

```

x = torch.tensor([1, 2, 3, 4, 5, 6])
split1, split2 = torch.split(x, 3) ①
print("분리된 텐서:", split1, split2)

```

1,2,3 4,5,6
3개씩 묶어 처리 tuple로 나옴. (split1, split2) 가능

OUTPUT

```

분리된 텐서: tensor([1, 2, 3]) tensor([4, 5, 6])

```

- ① 텐서 x를 길이가 3인 텐서로 각각 분할합니다. 결과적으로 길이가 6인 텐서 x는 길이가 3인 텐서 두 개로 나누어집니다. 텐서 x의 길이가 10이라면 길이가 3인 텐서 3개와 길이가 1인 텐서 하나, 즉 총 4개로 나누어집니다.

cat과 split 메서드는 실전에서 다양하게 사용됩니다. 예를 들어 연도별 판매 데이터를 하나로 합칠 때는 torch.cat 메서드를 사용할 수 있고, 하나의 텐서 데이터 세트를 학습용과 테스트용으로 나눌 때는 torch.split 함수를 사용할 수 있습니다. 다만 이 책은 실제 현장에서 더 자주 사용하는 방법인 sklearn의 train_test_split 함수를 활용해 텐서 데이터 세트를 나눕니다.

저장 신경망 학습 -> 순전파 학습

텐서 내적과 행렬곱
같은 뜻.

텐서 내적(Dot Product, 점곱)은 두 개의 1차원 텐서(벡터)를 곱해 하나의 숫자(스칼라)를 만드는 연산입니다.

```

CODE
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
dot = torch.dot(a, b)
print("Dot product:", dot)

OUTPUT
Dot product: tensor(32)

```

벡터

곱하기

벡터의 내적에는 dot 사용

마찬가지로 같은 위치의 값끼리(element-wise) 곱셈한 다음 모두 합쳐 계산합니다.

$\|a\|$: a 벡터의 길이

$$1 \times 4 + 2 \times 5 + 3 \times 6 = 4 + 10 + 18 = 32$$

a dot b = $\|a\| * \|b\| * \cos(\text{세타})$
내적

텐서 내적의 결과는 하나의 숫자인 스칼라(0차원 텐서)입니다. 딥러닝에서는 유사도 계산(Cosine Similarity)이나 노드 출력을 계산할 때 텐서 내적 연산을 사용합니다. 신경망에서 사용되는 내적값 자체가 유사도를 나타낼 수 있음. 신경망에 유사도 찾는 내용이 들어감.

유사도

$\cos(\text{세타}) = a \text{ dot } b / \|a\| * \|b\|$

-1 ~ 1 사이의 값

0에 가까우면 유사하지 않음.

행렬곱(Matrix Multiplication) 연산은 이해하면 좋겠지만, 조금 어려운 내용일 수 있으므로 편하게 넘어가도 괜찮습니다. 행렬곱 연산은 두 개의 다차원 텐서(행렬) 간의 특별한 연산입니다. 행렬 A와 행렬 B의 행렬곱 연산은 수학에서는 $A \circ B$ 로 표기합니다. 행렬곱의 예제 코드는 다음과 같습니다.

세타 = 90도 => 0 (영원히 다름)
세타 = 0도 => 1 (같음)

```

CODE
A = torch.tensor([[1, 2], [3, 4]])
B = torch.tensor([[5, 6], [7, 8]])
result = torch.matmul(A, B)
print(result)

```

2차

2차

① 행렬 내적에는 matmul 사용

2 X 2

2 X 2

가능. 결과는 2 X 2 로 나옴.

```

OUTPUT
tensor([[19, 22],
        [43, 50]])

```

1 2

3 4

5 6

7 8

① 행렬 A와 행렬 B의 행렬곱 연산(matmul)을 수행합니다.

행렬곱 연산은 행렬의 순서가 중요합니다. $A \circ B$ 라고 하면 행렬 A의 행과 행렬 B의 열을 내적하여 값을 구합니다. 예를 들어 A의 1행 [1, 2]와 B의 1열 [5, 7]을 내적하면 $1 \times 5 + 2 \times 7 = 19$ 가 되고, 이 값이 결과 행렬의 1행 1열에 위치합니다. 이렇게 모든 행과 열에서 내적 연산을 수행해 최종적인 결과 행렬을 출력합니다.

$$\begin{bmatrix} 1 \times 5 + 2 \times 7, & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7, & 3 \times 6 + 4 \times 8 \end{bmatrix} = \begin{bmatrix} 19, & 22 \\ 43, & 50 \end{bmatrix}$$

신경망 내부는
행렬과 행렬의 내적

행렬곱 연산을 할 때는 torch.matmul 함수 대신 @ 연산자를 사용할 수도 있습니다.

- 철수, 영희 과일가게에서 사과, 바나나 구매.
 구매 수량
 철수: 사과 3개, 바나나 2개
 영희: 사과 1개, 바나나 4개

과일 가격
 사과: 1000원
 바나나: 1500원

$$\begin{pmatrix} 3 & 2 \\ 1 & 4 \end{pmatrix} \times \begin{pmatrix} 1000 \\ 1500 \end{pmatrix} = \begin{pmatrix} 6000 \\ 7000 \end{pmatrix}$$

`torch.matmul(A, B)`와 `A@B`의 결과는 동일합니다.

gpu는 병렬로 연산이 됨.
 cuda는 내적이 주 기능

행렬곱 연산을 수행할 때는 주의할 점이 있습니다. A의 열 수와 B의 행 수가 일치해야 합니다. 예를 들어 A가 2×3 행렬이고 B는 3×4 행렬이면 `A@B`의 연산이 가능합니다. A의 열 수인 3과 B의 행 수인 3이 같기 때문입니다. 그러나 `B@A` 연산은 B의 열 수 4와 A의 행 수 2가 불일치하므로 연산할 수 없습니다.

행렬곱 연산은 다소 어렵습니다. 하지만 걱정할 필요는 없습니다. 딥러닝 코드를 작성하면서 직접 행렬곱을 계산할 일은 거의 없습니다. 다만 개념을 이해해 두면 이후 더 깊이 공부할 때 큰 도움이 됩니다.

지금까지 텐서를 다루는 몇 가지 방법을 살펴보았습니다. 이어서 텐서를 기반으로 학습할 때 데이터를 배치 단위로 공급하는 데이터 로더를 설명합니다.

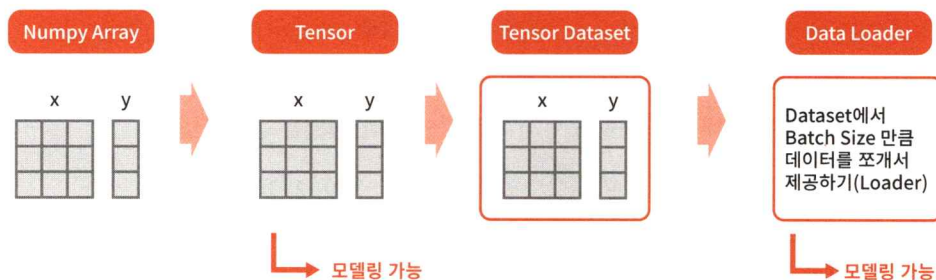
데이터 로더

딥러닝 모델을 학습할 때는 단순히 데이터를 배열이나 텐서 형태로 불러오는 것만으로는 충분하지 않습니다. 모델을 효율적으로 학습시키려면 데이터를 무작위로 섞고(Shuffle) 일정 크기의 미니배치(Mini Batch) 단위로 나누고 필요할 때마다 자동으로 불러오는 기능이 필요합니다. 이를 위해 데이터 로더(DataLoader)를 사용합니다.

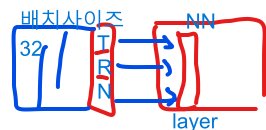
배치(Batch)란 ‘묶음’을 의미합니다. 딥러닝은 데이터를 하나씩 순차적으로 학습하는 방식이 아니라, 여러 개를 묶어 동시에 처리합니다. 이때 지나치게 큰 묶음이 아닌 작은 단위로 데이터를 나누어 학습하는데, 이 단위를 미니배치(Mini Batch)라고 합니다.

데이터 로더는 데이터를 미니배치 단위로 나누어 모델에 공급함으로써 데이터를 효율적으로 학습할 수 있도록 돕습니다.

데이터 로더를 만들기 위한 절차는 [그림 2-4]와 같습니다.



[그림 2-4] 넘파이 배열부터 데이터 로더까지 만드는 절차



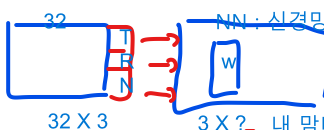
들어가는
건 벡터 신경망 안쪽은
 행렬

so, 배치로
행렬을
만들어줌



행렬곱 가능

배치 사이즈 관련.



32 X 3 3 X ? 내 맘대로

단계가 많아 보이지만 코드로 구현하면 간단합니다. '2일차_텐서연습.ipynb' 파일에서 '광고비에 따른 매출액' 데이터(Advertising)를 불러와 데이터 로더를 구현하겠습니다.

넘파이 배열→텐서 변환

'2일차_텐서연습.ipynb'의 마지막 빈 셀에서 데이터를 불러와 넘파이 배열을 준비합니다.

CODE

```
path = 'https://bit.ly/advertising_csv'
adv = pd.read_csv(path)
target = 'Sales'
x = adv.drop(target, axis=1)
y = adv.loc[:, target]
x_np = x.values      ①
y_np = y.values
print(type(x_np))    ②
print(x_np.shape)
```

OUTPUT

```
<class 'numpy.ndarray'>
(200, 3)
```

- ① 판다스의 데이터프레임 혹은 시리즈 뒤에 values 속성을 붙이면 넘파이 배열이 됩니다.
- ② type 함수로 x_np의 타입을 확인합니다.

데이터를 불러와 x, y로 분할한 다음 넘파이 배열을 만들었습니다. x_np의 크기는 200행 3열입니다.

다음으로 넘파이 배열을 텐서로 변환합니다. 이때 데이터는 실수 타입으로 지정합니다

CODE

```
# 넘파이 배열 x, y를 텐서로 변환
x_tensor = torch.tensor(x_np, dtype=torch.float32)
y_tensor = torch.tensor(y_np, dtype=torch.float32)
print(x_tensor.shape, y_tensor.shape)
```

OUTPUT

```
torch.Size([200, 3]) torch.Size([200])
```

넘파이 배열의 크기 그대로 텐서로 변환했습니다. x_tensor와 y_tensor의 데이터는 각각 200건입니다.

넘파이 배열과 파이토치 텐서는 모양이 비슷해 보이지만 중요한 차이가 있습니다. 넘파이 배열은 계산만 담당하는 반면, 파이토치 텐서는 GPU 연산과 자동 미분(AutoGrad)을 지원합니다. 자동 미분은 연산 과정을 기록해 두었다가 역전파 단계에서 기울기를 자동으로 계산해 주는 기능입니다. 이 기능 덕분에 우리는 복잡한 미분 계산을 직접 하지 않고도 모델을 학습시킬 수 있습니다. 그래서 일반 연산에는 넘파이를 주로 쓰지만, 딥러닝 학습에서는 반드시 텐서를 사용해야 합니다.

x 텐서, y 텐서 → 텐서 데이터 세트 변환

이번에는 x 텐서, y 텐서를 하나로 묶어 텐서 데이터 세트로 변환합니다.

CODE

```
from torch.utils.data import DataLoader, TensorDataset ①
dataset = TensorDataset(x_tensor, y_tensor) ②
print(dataset[:3])
print(len(dataset))
```

OUTPUT

```
(tensor([[230.1000, 37.8000, 69.2000],
        [ 44.5000, 39.3000, 45.1000],
        [ 17.2000, 45.9000, 69.3000]]), tensor([22.1000, 10.4000, 9.3000]))
200
```

- ① torch.utils.data 모듈에 있는 함수 DataLoader와 TensorDataset을 불러옵니다.
- ② TensorDataset 함수에 x_tensor와 y_tensor를 입력하고 텐서 데이터 세트를 생성합니다. 생성한 데이터 세트를 변수 dataset에 저장합니다.

텐서 데이터 세트를 만들 때 주의할 점이 하나 있습니다. x_tensor의 데이터 건수와 y_tensor의 데이터 건수가 같아야 합니다. 앞에서 텐서로 변환하고 구조(shape)를 확인했을 때 x_tensor와 y_tensor 모두 200건씩 같은 크기였습니다.

텐서 데이터 세트에서 print(dataset[:3])으로 상위 3건의 데이터를 확인하면 2차원 텐서(x_tensor) 3건과 1차원 텐서(y_tensor) 3건을 출력합니다. len 함수로 텐서 데이터 세트의 크기를 확인하면 데이터가 총 200건 있음을 알 수 있습니다.

텐서 데이터 세트로 데이터 로더 선언

생성한 텐서 데이터 세트를 이용해 데이터 로더를 선언합니다.

CODE

```
# 데이터 로더 생성
data_loader = DataLoader(dataset, batch_size = 32, shuffle = True)
```

`DataLoader` 함수에서 `shuffle` 옵션을 `True`로 지정하면 데이터를 무작위로 섞습니다. 또한 `batch_size=32`는 배치 크기를 32로 지정하는데, 배치 하나에 데이터가 32건씩 담기도록 합니다. 전체 데이터 건수가 200건이고 배치 크기가 32이므로 총 7개의 배치가 만들어집니다.



[그림 2-5] 200건의 데이터를 크기 32로 나눌 때의 배치 예시

[그림 2-5]를 보면 앞에서부터 6개의 배치에는 데이터가 32건씩 담기고 마지막 배치에는 나머지 8건이 담깁니다.



배치 크기는 마음대로 정해도 되나요?

배치 크기의 기본값은 32입니다. 이는 32건의 데이터(2차원 데이터 세트라면 32행의 데이터)를 한 묶음(배치)으로 만들어 학습시킨다는 의미입니다. 전체 학습 데이터 세트가 크면(데이터 건수가 많으면) 배치 크기를 더 크게 잡고, 데이터가 적으면 좀 더 작은 값으로 지정할 수 있습니다. 배치 크기는 학습을 시키면서 조절하는 하이퍼파라미터이므로 정답이 있는 것은 아닙니다. 그러나 기본값인 32를 많이 사용합니다.

선언한 데이터 로더에서 반복문(for loop)으로 배치를 하나씩 꺼내 배치가 어떤 형식으로 되어 있는지(shape) 확인하겠습니다.

CODE

```
for n, batch in enumerate(data_loader): ①
    x_batch, y_batch = batch ②
    print(f"배치 {n}, x 배치 크기:{x_batch.shape}") ③
```

OUTPUT

```
배치 0, x 배치 크기: torch.Size([32, 3])
배치 1, x 배치 크기: torch.Size([32, 3])
배치 2, x 배치 크기: torch.Size([32, 3])
배치 3, x 배치 크기: torch.Size([32, 3])
배치 4, x 배치 크기: torch.Size([32, 3])
배치 5, x 배치 크기: torch.Size([32, 3])
배치 6, x 배치 크기: torch.Size([8, 3])
```

① 데이터 로더(`data_loader`)에서 배치를 하나씩 불러와 변수 `batch`에 저장합니다. 이때 `enumerate` 함수를 사용하면 반복문의 순번호 함께 제공합니다(순번호는 `n`에 저장).

② 배치 하나에는 `x` 배치와 `y` 배치가 있습니다.

③ 배치의 순번호와 `x` 배치의 크기를 출력합니다.

각각의 배치를 불러와 배치에 담긴 데이터의 건수를 확인했습니다. 모델 학습 과정에서 데이터 로더를 사용하면 배치 단위로 학습(오차의 순전파와 역전파)이 이루어집니다.

디바이스 할당: GPU 혹은 CPU에 할당하기

딥러닝 모델을 학습할 때는 모델이나 데이터를 디바이스에 할당합니다. 여기서 디바이스는 연산을 처리하기 위한 GPU나 CPU를 의미합니다. 디바이스를 할당한다는 말은 데이터 처리 또는 모델의 학습 연산을 어느 장치에서 수행할지 정한다는 의미입니다.

딥러닝에서 GPU가 필요한 이유

디바이스를 적절히 설정하는 일은 모델의 학습 속도와 성능에 중요한 영향을 미칩니다. 딥러닝 학습에서는 수많은 행렬 연산(텐서 연산)이 필요한데, 이 과정에서 GPU의 역할은 매우 중요합니다. GPU가 필요한 이유는 다음과 같습니다.

- 대규모 병렬 연산 처리 능력: GPU는 수천 개의 작은 코어로 구성되어 있어 여러 개의 연산을 동시에 수행하는 데 최적화되어 있습니다. 딥러닝에서는 행렬 연산이 매우 빈번하게 사용되기 때문에 GPU는 이러한 연산을 병렬로 처리함으로써 학습 속도를 극대화합니다. 반면 CPU는 직렬(Sequential) 처리가 강점이므로 데이터 전처리와 같은 작업에 적합합니다.
- 고성능 학습을 위한 메모리 대역폭: GPU는 CPU보다 메모리 대역폭이 훨씬 커 대량의 데이터를 빠르게 로딩하고 계산합니다. 따라서 GPU를 사용하면 일반적으로 CPU보다 10배에서 100배 이상 빠르게 모델을 학습시킬 수 있습니다. 특히 대량의 데이터 세트와 복잡한 신경망 구조(CNN, RNN 등)를 학습시킬 때 GPU의 사용은 필수적입니다.

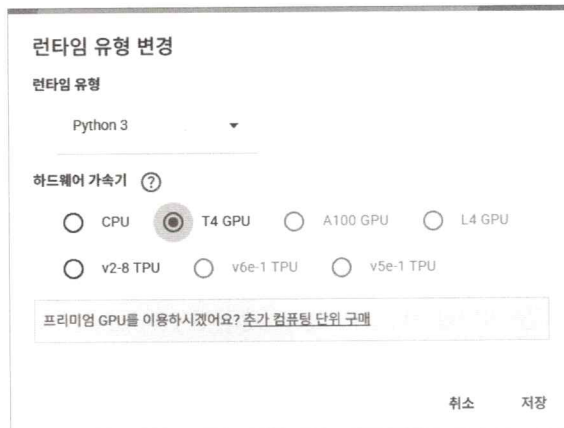
하지만 연산을 모두 GPU로 처리하는 것은 효율적이지 않을 수도 있습니다. GPU 자원은 상대적으로 CPU에 비해 비싸기 때문입니다. 따라서 실무에서는 GPU와 CPU를 적절히 혼용해 사용합니다. CPU는 주로 데이터의 전처리나 학습된 모델로 예측을 수행할 때 사용합니다. 별도의 디바이스를 지정하지 않으면 기본값은 CPU입니다. 반면 GPU는 모델 학습과 검증 연산에 사용합니다. CPU와 달리 GPU는 모델과 데이터 세트를 명시적으로 할당해야 이 디바이스를 사용할 수 있습니다.

디바이스 할당하기

장치관리자 -> 성능 -> GPU
첫 번째 3D -> Cuda 변경

다시 새로운 코랩 파일을 생성하고 이름은 '2일차_디바이스할당.ipynb'로 합니다.
코랩을 작성하기 전에 먼저 런타임 유형을 변경해야 합니다.

코랩 상단 메뉴에서 [런타임] - [런타임 유형 변경]을 차례로 선택합니다. [런타임 유형 변경] 대화상자가 나타납니다. 대화상자에서 [그림 2-6]과 같이 GPU를 'T4 GPU'로 선택하고 <저장> 버튼을 클릭합니다.



[그림 2-6] [런타임 유형 변경] 대화상자에서 GPU 선택

필요한 라이브러리와 '광고비에 따른 매출액' 데이터를 불러와 데이터를 준비합니다.

CODE

```
# 라이브러리 로딩
import torch
from torch import nn
import numpy as np
import pandas as pd

# 데이터 로딩 및 x, y 분할
path = 'https://bit.ly/advertising_csv'
adv = pd.read_csv(path)

target = 'Sales'
x = adv.drop(target, axis=1)
y = adv.loc[:, target]

# numpy로 변환한 다음 텐서로 변환
x_np = x.values
```

```
y_np = y.values
x_tensor = torch.tensor(x_np, dtype=torch.float32)
y_tensor = torch.tensor(y_np, dtype=torch.float32)
```

이제 코드로 CPU와 GPU 디바이스의 이름을 준비합니다.

CODE

```
device = "cuda" if torch.cuda.is_available() else "cpu" ①
print(f"Using {device} device.") ②
```

OUTPUT

Using cuda device.

- ① torch.cuda.is_available()로 현재 GPU를 사용할 수 있는 환경인지 확인합니다. 사용 가능하다면 변수 device에는 cuda를, 그렇지 않으면 cpu를 저장합니다.
- ② device를 출력해서 저장된 값을 확인합니다.

여기서 CUDA(Compute Unified Device Architecture)는 엔비디아(NVIDIA)에서 제공하는 GPU 기술을 의미합니다. 코드를 실행하면 GPU를 사용할 수 있는 cuda가 출력됩니다.

모델과 학습 데이터를 디바이스에 할당하려면 다음과 같이 코드를 작성합니다.

CODE

```
# 모델을 디바이스(GPU)에 할당하기
model = nn.Sequential( nn.Linear(3, 1) ) ①
model = model.to(device) ②
```

데이터 세트를 디바이스에 할당.

```
x, y = x_tensor.to(device), y_tensor.to(device) ③
```

- ① 모델을 선언합니다. 모델링에 대한 자세한 설명은 3일 차에서 다룹니다. 여기서는 “아주 간단한 딥러닝 모델은 이런 식으로 선언하는구나”라고 생각하고 넘어갑시다.
- ② to(device) 메서드로 선언한 모델을 GPU로 이동시킵니다. to(device)를 지정하지 않으면 기본적으로 CPU에 할당됩니다.
- ③ to(device) 메서드로 데이터 세트 역시 GPU로 이동시킵니다. 역시 디바이스 할당 작업을 하지 않으면 기본값은 CPU입니다.

모델과 데이터 세트를 디바이스에 할당할 때는 모두 to(device) 메서드를 사용합니다. 한 가지 중요한 점은 모델과 데이터가 반드시 같은 디바이스에 있어야 연산이 가능하다는 것입니다. 즉, 둘 다 GPU에 할당되거나 둘 다 CPU에 할당되어야 오류 없이 동작합니다.

디바이스 위치 옮기기

디바이스 할당에서 한 가지 더 알아야 할 것이 있습니다. 현재 디바이스를 다른 디바이스로 옮기는 방법입니다. 예를 들면 cuda의 데이터를 cpu로 옮겨야 할 수도 있고, 반대의 경우도 발생할 수 있습니다.

먼저 현재 디바이스를 확인하는 코드는 다음과 같습니다.

CODE

```
print(next(model.parameters()).device) ①  
print(x.device, y.device)              ②
```

OUTPUT

```
cuda:0  
cuda:0 cuda:0
```

- ① `next(model.parameters())`는 모델에서 학습 가능한 모든 파라미터(가중치)를 반환합니다. 이때 이 파라미터들이 어느 디바이스에 있는지 device 속성으로 확인합니다.
- ② 학습할 데이터 세트의 디바이스도 함께 확인합니다.

실행 결과를 보면 모델 파라미터와 데이터 세트(x, y)가 모두 `cuda:0`으로 출력됩니다. 모두 GPU에 할당된 상태를 의미합니다. 디바이스 이름 뒤의 `:0`은 GPU의 순번을 나타내는데, 만약 GPU가 두 개 이상이라면 원하는 디바이스 번호를 지정해 특정 GPU에 할당할 수도 있습니다.



모델이 어느 디바이스에 있는지 확인할 때 왜 파라미터를 확인할까?

디바이스 할당은 연산을 CPU에서 수행할지 GPU에서 수행할지 정하는 과정입니다. 딥러닝 모델은 여러 개의 파라미터(가중치 텐서)로 구성되어 있는데, 학습이나 추론할 때는 이 파라미터들이 입력 데이터와 함께 연산에 사용됩니다. 그런데 파이토치의 모델 객체 자체에는 device 속성이 없습니다. 따라서 모델의 디바이스를 확인하려면 내부 파라미터 중 하나를 꺼내어 그 파라미터의 device 속성을 확인해야 합니다. 결국 파라미터가 할당된 디바이스가 곧 모델이 실제로 연산을 수행할 장소를 의미합니다.

GPU와 CPU 간에 할당된 디바이스를 전환하려면 다음과 같은 코드를 이용합니다.

CODE

```
# GPU -> CPU  
x_cpu = x.cpu() ①  
y_cpu = y.to("cpu")  
print(x_cpu.device, y_cpu.device)  
# CPU -> GPU  
x_gpu = x_cpu.cuda() ②
```

```
y_gpu = y_cpu.to("cuda")
print(x_gpu.device, y_gpu.device)
```

OUTPUT

```
cpu cpu
cuda:0 cuda:0
```

- ① 현재 x는 cuda로 GPU에 할당되어 있는 상태인데, x.cpu 또는 x.to('cpu') 메서드를 사용하면 x를 CPU로 옮기고 변수 x_cpu에 저장합니다.
- ② x_gpu = x_cpu.cuda는 x를 다시 GPU로 옮기는 명령입니다. cuda 대신 to('cuda') 메서드를 사용할 수도 있습니다.

이처럼 디바이스를 전환하는 코드는 간단합니다.

첨언하면 파이토치에서 제공하는 함수들은 CPU와 GPU에 할당된 데이터를 모두 처리할 수 있습니다. 하지만 sklearn이나 numpy 같은 라이브러리 함수들은 GPU 연산을 지원하지 않으며 파이토치 텐서를 직접 입력값으로 받을 수도 없습니다. 예를 들어 sklearn의 mean_absolute_error 함수는 넘파이 배열, 판다스 시리즈(Series), 리스트(List)와 같은 1차원 데이터 구조만 받을 수 있기 때문에 1차원 텐서는 인식하지 못합니다. 따라서 sklearn과 같은 함수를 사용할 때는 GPU에 있던 텐서를 CPU로 옮기고 다시 넘파이 배열로 변환해야 합니다.

- CUDA→CPU: 최종 형태가 넘파이 배열입니다. 넘파이 배열은 cuda를 지원하지 않기 때문에 cpu로 옮겨야 합니다.
- Tensor→numpy 배열: 텐서를 넘파이 배열로 변환합니다.

이를 한꺼번에 변경할 수 있는 코드를 적으면 다음과 같습니다. 예시이니 직접 따라할 필요는 없습니다.

CODE

```
pred = pred.cpu().numpy() ①
y = y.cpu().numpy()
mean_absolute_error(y, pred)
```

- ① cpu 메서드를 사용해 CPU로 디바이스를 변경하고, numpy 메서드를 사용해 넘파이 배열로 변환합니다.

지금까지 딥러닝을 이해하는 데 꼭 필요한 텐서와 데이터 로더 그리고 디바이스 할당을 설명했습니다. 내용이 좀 길었지만, 딥러닝 모델링의 기본기를 닦는 부분이니 이해가 안 되는 부분이 있다면 다시 한번 꼼꼼히 읽어 보기를 권합니다.

1. 오차 min

오차 min인

GD : 경사하강법 => weight 구하기

2. weight update(수정)

학습

딥러닝 학습 과정 이해 2: 가중치 조정

LLM에서는 가중치를
파라미터라고 함.

이번 차시의 마지막 절에서는 가중치 조정을 살펴봅니다. 딥러닝 모델 학습에서 가장 중요한 요소 중 하나가 바로 **가중치(Weight)**입니다. 데이터에는 다양한 패턴이 숨어 있는데, 학습 과정에서 이 패턴을 수치로 표현한 것이 가중치입니다. 가중치는 데이터의 규칙과 특징을 담아낸 숫자입니다. 따라서 **모델 학습의 핵심 목표**는 **오차 최소** ~ **최적의 가중치를 찾는 일**입니다. 이번 절에서는 가중치가 무엇인지 어떻게 조정되는지 그리고 딥러닝에서 가중치 학습이 어떤 방식으로 이루어지는지 이해하는 데 초점을 맞추겠습니다.

가중치란?

간단한 예를 들어 가중치(Weight)를 설명하겠습니다. 어떤 매장에서 1~3월의 판매량 데이터로 4월의 판매량을 예측하려고 합니다.

[표 2-1] 특정 매장의 월별 판매량 합계

1월 판매량	2월 판매량	3월 판매량	4월 판매량
110	120	125	?
10	5	2.5	

어떻게 하면 4월의 판매량을 예측할 수 있을까요? 가장 간단한 방법은 1~3월 판매량의 평균을 계산해 보는 것입니다.

$$4\text{월 판매량} = \frac{1}{3} \cdot 1\text{월 판매량} + \frac{1}{3} \cdot 2\text{월 판매량} + \frac{1}{3} \cdot 3\text{월 판매량}$$

이 식은 1~3월의 판매량으로 4월의 판매량을 예측하는 수식인데, 1, 2, 3월 각각의 판매량이 1/3씩 비중이 똑같다는 의미로도 해석할 수 있습니다. 다른 말로 표현하면 1, 2, 3월 판매량의 **가중치(w)**가 모두 1/3이라는 뜻입니다.

$$4\text{월 판매량} = w \cdot 1\text{월 판매량} + w \cdot 2\text{월 판매량} + w \cdot 3\text{월 판매량}, w = \frac{1}{3}$$

이제 다르게 접근해 봅시다. 4월 판매량을 더 정확하게 예측하려면 어떻게 해야 할까요? 1~3월의 판매량을 모두 똑같은 가중치로 봐야 할까요? 아닐 수도 있겠다는 생각이 들죠? 어떤 사람은 3월에 가장 큰 가중치를, 다음으로 2월, 그리고 1월에 가장 작은 가중치를 부여해야 한다고 생각할 수 있습니다. 타당한 의견입니다. 가중치를 다르게 설정해야 더 정확한 예측이 가능할 수 있습니다.

$$4\text{월 판매량} = w_1 \cdot 1\text{월 판매량} + w_2 \cdot 2\text{월 판매량} + w_3 \cdot 3\text{월 판매량}$$

가중치는 입력 데이터(1, 2, 3월의 판매량)의 영향력을 조정하는 값입니다. 그렇다면 가중치 w_1, w_2, w_3 를 어떻게 정해야 할까요? 가장 적절한 가중치는 어떻게 찾을 수 있을까요?

가중치 업데이트

가중치 업데이트 자체가 학습 =>

$$w_t = w_{t-1} - \alpha \cdot \text{기울기 (미분)}$$

w_t : 현재 w_{t-1} : 바로 이전 α : 학습률 * 기울기 (미분)

가중치 업데이트 절차

이제 적절한 가중치를 찾는 중요한 방법을 소개하려고 합니다. 처음부터 정답(최적의 가중치)을 찾을 수는 없으니, 일단 첫 발을 떼고 한 걸음씩 움직이면서 목적지인 최적의 가중치에 접근해 봅시다. 절차는 다음과 같습니다.

- ① 초기 가중치를 임의로 설정: 가중치의 초기값은 랜덤한 값으로 할당합니다.
- ② 현재의 가중치로 예측값 계산: 현재의 가중치로 구성된 모델에 학습 데이터를 입력하고 예측 결과를 계산합니다.
- ③ 목표값과 차이(오차) 계산: 예측값이 목표값과 얼마나 차이가 나는지 확인합니다. 이때 손실 함수에서 지정한 오차 계산 방식을 이용합니다. ②~③의 과정을 오차의 순전파라고 했습니다.
- ④ 오차를 줄이는 방향으로 가중치 조정: 오차를 줄이려면 가중치를 조정해야 합니다. 그렇다면 어느 방향으로 가중치를 조정해야 할까요? 또 얼마나 조정해야 할까요? 이때 가중치를 조절하는 해결사가 바로 옵티마이저(Optimizer)입니다. 옵티마이저는 가중치를 높일지 낮출지를 결정하고 얼마나 조정할지 찾아 적용하는 역할을 합니다. 손실 함수로 오차를 계산하고 옵티마이저로 가중치를 조정합니다. 조정된 가중치는 다시 모델 식에 적용됩니다. 이 과정을 오차의 역전파라고 했습니다.
- ⑤ 이전 과정(②~④)을 반복: 지정한 횟수(에포크)만큼 반복 학습합니다. 반복 학습하며 오차를 관찰하는 과정에서 오차가 거의 줄어들지 않으면 학습을 멈추도록 설정할 수도 있습니다. 보통 에포크마다 오차를 측정하고 오차가 줄어드는지 학습 곡선을 그려 학습이 적절히 이루어지고 있는지 판단합니다.

학습의 첫 단계에서 가중치는 무작위 값으로 초기화되기 때문에 이 책의 학습 결과와 여러분의 결과가 조금씩 다를 수 있습니다. 또한 동일한 학습 코드를 여러 번 실행

행하더라도 매번 결과가 달라질 수 있습니다. 이런 차이는 자연스럽게 발생하는 변동이므로 그대로 받아들이면 됩니다.

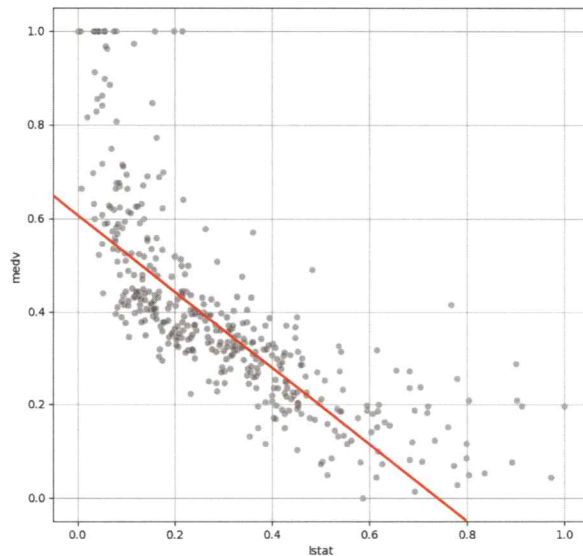
가중치 업데이트 예시

딥러닝 가중치 업데이트 절차를 보스턴 집값 데이터를 예시로 다시 한번 살펴보겠습니다.

보스턴 집값 데이터(Boston Housing Dataset)는 1970년대 보스턴 지역의 주택 가격을 설명하기 위해 수집된 데이터 세트로서 하위 계층 비율, 방 개수, 범죄율, 교육 수준 등 여러 특징(feature)을 포함하고 있으며 집값(중위 가격)을 예측하는데 널리 활용됩니다.

가중치 업데이트의 학습 절차를 확인하는 예제이므로 여기서는 하위 계층 비율(변수명: `lstat`)로 집값(변수명: `medv`)을 예측하는 간단한 모델을 만들어 설명합니다.

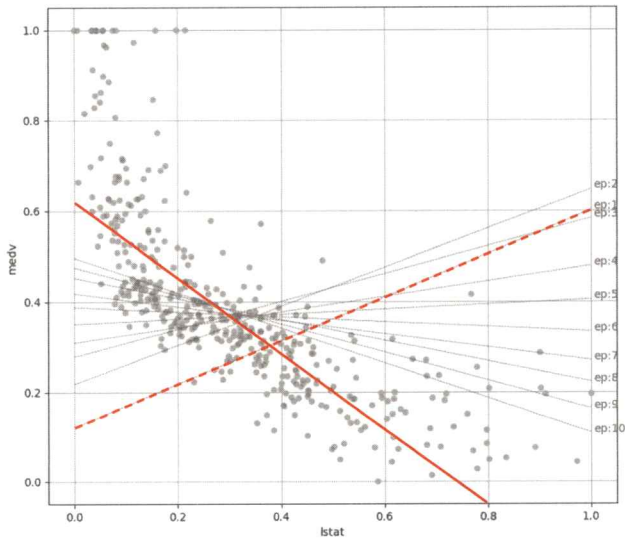
[그림 2-7]의 빨간색 선에 대한 식은 $medv = w_1 \cdot lstat + w_0$ 과 같습니다. 일반적으로 w_1 을 기울기, w_0 을 절편이라고 합니다. 이 두 값이 가중치입니다. 그렇다면 최적의 가중치는 어떻게 찾을까요? 앞에서 가중치 업데이트를 설명한 순서대로 짚어 보겠습니다.



[그림 2-7] 보스턴의 집값 데이터. 하위 계층 비율과 집값에 대한 산점도

- ① 초기 가중치를 임의로 설정: $medv = 0.5 \cdot lstat + 0.17$. 여기서 0.5와 0.17은 랜덤하게 지정한 값입니다.
- ② 현재 가중치로 예측값 계산: 만약 $lstat$ 이 0.4라면 집값($medv$)에 대한 예측값은 $0.5 \times 0.4 + 0.17 = 0.37$ 입니다. 이런 식으로 학습 데이터를 모델에 입력하고 예측값을 모두 계산합니다.
- ③ 목표값과의 차이(오차) 계산: 예측값과 목표값의 차이를 계산합니다. 집값을 예측하는 회귀 모델이므로 손실 함수는 MSE(Mean Squared Error)를 사용합니다.
- ④ 오차를 줄이는 방향으로 가중치 조정: 옵티마이저가 오차를 줄이는 방향과 조절량을 결정해 0.5를 0.4로, 0.17를 0.21로 조정한다고 가정해 봅시다. 그러면 모델 식은 $medv = 0.4 \cdot lstat + 0.21$ 이 됩니다.
- ⑤ 이 과정(②~④)을 반복합니다.

가중치를 조절해 변동되는 모델의 예시 그래프는 [그림 2-8]과 같습니다.



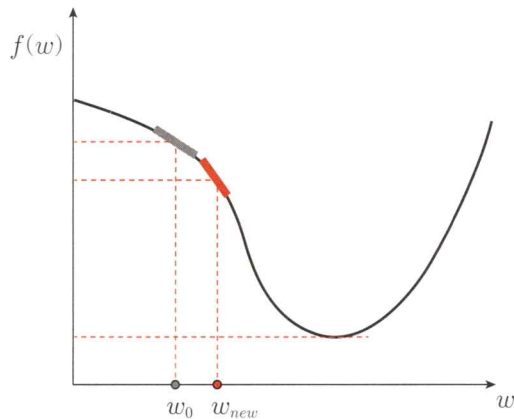
[그림 2-8] 가중치 조정에 따른 모델 식의 변화

최종 모델은 [그림 2-8]에서 보이는 굵은 빨간색 선입니다. 여러 개의 얇은 선은 가중치(기울기와 절편)를 조정할 때마다 바뀌는 모델 식입니다. 그래프 오른쪽 끝의 ep#은 에포크 1에서 에포크 10까지를 나타냅니다. 조금 굵은 빨간색 점선(---)은 가중치의 초기값(#ep1) 모델입니다. 가중치가 조금씩 조정되면서 모델이 점차 최종 모델(굵은 선)에 가까워지고 있습니다.

경사 하강법

가중치 조정은 옵티마이저의 역할이라고 했습니다. 최근 딥러닝에서는 대표적인 옵티마이저로 Adam을 주로 사용합니다. Adam의 기본 원리는 경사 하강법(Gradient Descent)에 기초하고 있습니다. 경사 하강법은 손실 함수를 줄이기 위해 가중치를 어느 방향으로(+ 또는 -) 이동시키고 얼마나 크게 조정할지 결정하는 방법입니다.

[그림 2-9]를 보면서 경사 하강법을 설명하겠습니다. 그래프의 x축은 조정할 가중치(w), y축은 손실 함수(Loss Function)의 결과, 즉 오차입니다.



[그림 2-9] 경사 하강법

단계별로 설명하면 다음과 같습니다.

1. 가중치의 초기값 설정: 학습은 먼저 임의의 시작점 w_0 (초기 가중치)을 정하는 일에서 시작합니다. 초기 가중치 값은 대부분 랜덤하게 초기화되는데, 아직은 오차가 큰 상태입니다. 여기서는 오차, 즉 그래프에서는 $f(w)$ 의 값을 최대한 줄일 수 있도록 w 를 이동시키는 것이 목표입니다.
2. 기울기(Gradient)를 계산: 이제 시작점에서 어느 방향으로 이동해야 오차가 줄어드는지 알아야 합니다. 이때 사용하는 것이 바로 기울기, 즉 그래디언트(Gradient)입니다. 그래디언트는 미분값입니다.
 - 기울기가 양수라면 w 를 왼쪽(- 방향)으로 조정해야 오차가 줄어듭니다.
 - 기울기가 음수라면 w 를 오른쪽(+ 방향)으로 조정해야 오차가 줄어듭니다.

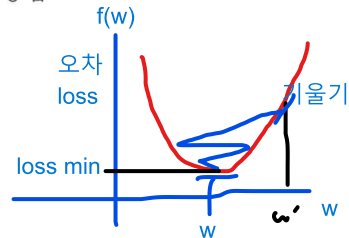
항상 기울기의 반대 방향으로 가중치를 이동시키는 것이 핵심입니다. [그림 2-9]에서 w_0 (초기 가중치)의 기울기는 음수(-)입니다. 그러므로 w 를 오른쪽 방향으로 조정해야 합니다. 즉, w_0 을 약간 증가시켜야 합니다.

3. 조절량 조정 - 학습률(Learning Rate): 기울기 방향은 알았지만, 얼마나 이동시킬지도 중요합니다. 너무 많이 움직이면 최적값을 지나갈 수 있고, 너무 조금 움직이면 학습 속도가 느려질 수 있습니다. 이 이동의 크기를 결정하는 값이 바로 학습률(Learning Rate)입니다. 보통 학습률의 기호는 η (에타, eta)를 사용합니다. 학습률을 적용한 수식은 다음과 같습니다.

$$w_{new} = w_0 - \eta \times \frac{\partial f(w)}{\partial w}$$

w_0 ← w 에 대한 함수
 η ← w 로 미분해라

- w_{new} : 업데이트된 새로운 가중치
- w_0 : 현재의 가중치
- η : 학습률(Learning Rate)
- $\frac{\partial f(w)}{\partial w}$: 손실 함수에 대한 미분값(기울기, 그래디언트)



최적의 w 찾기
 학습
 최적화 => Optimizer

이 수식대로 기울기를 따라 w 를 조금씩 이동시키면 오차는 점차 줄고 모델은 더 좋은 예측을 하게 됩니다. 여기서 기울기에 학습률을 곱한 값이 바로 조절량이고 기울기의 반대 방향이 조정되는 방향입니다.

4. 반복→수렴(Convergence): 이 과정을 반복적으로 수행하며 가중치를 조금씩 업데이트하면 결국 한곳으로 수렴합니다. 이때 수렴하는 지점에서 오차가 가장 작아지는 최적의 가중치가 나옵니다. 실제로 딥러닝에서는 이 과정을 수백, 수천 번 반복(Epoch)해 오차를 점차 줄입니다. 이런 원리로 가중치를 조정하는 방식이 바로 경사 하강법입니다.



Adam 옵티마이저란?

Adam 옵티마이저는 딥러닝에서 가장 많이 사용하는 최적화 알고리즘 중 하나입니다. Adam 옵티마이저는 기본적인 경사 하강법(Gradient Descent)에 두 가지 개념을 결합해 보다 **빠르고**

안정적으로 가중치를 업데이트할 수 있도록 고안한 방법입니다.

Adam은 다음 두 가지 정보를 함께 사용합니다

← 속도 빠름
 안정적 adam은 global minimum 잘 찾음

- 1차 모멘트(Momentum): 기울기(Gradient)를 누적해 방향을 안정적으로 유지하도록 합니다. 쉽게 말하면 방향성에 있어서는 최근의 경향을 따라가는 방식입니다.
- 2차 모멘트(RMSprop): 기울기의 제곱 평균을 이용해 기울기가 큰 방향은 작게, 작은 방향은

크게 조정합니다. 그럼 스케일을 자동으로 조절하는 효과가 있습니다.

왜 Adam을 많이 사용할까요? 첫째, Adam은 반복할 때마다 학습률을 다르게 조정하므로 최적의 성능을 찾는 데 좀 더 용이합니다. 둘째, 빠르게 수렴하면서도 진동이나 틱이 적어 안정적인 학습이 가능합니다. 셋째, 딥러닝에서는 데이터를 작은 묶음(미니배치)으로 나누어 학습하는데, 이때 매번 계산되는 기울기가 조금씩 달라져 학습이 불안정해질 수 있습니다. Adam은 이런 변화를 잘 조정해 주기 때문에 작은 단위로 나누어 학습하더라도 빠르고 안정적으로 좋은 성능을 낼 수 있습니다. 따라서 최근 딥러닝 모델의 개발자 대다수는 Adam 옵티마이저를 사용합니다.

정리

이번 차시에서는 딥러닝 코드를 무작정 따라해 보면서 딥러닝의 구조(흐름)를 살펴보았습니다. 더불어 딥러닝에서 사용하는 데이터 형식인 텐서와 가중치 조정 등을 상세히 설명했습니다.

딥러닝 코드의 흐름은 머신러닝과 유사한 점도 있지만 몇 가지 지점에서 차이가 있습니다. 먼저 딥러닝에서는 데이터 전처리 파트에서 스케일링을 필수로 적용해야 하고, 학습 데이터 세트를 텐서 데이터 세트로 만들고 데이터 로더로 선언해야 합니다. 또한 모델링 파트에서는 모델의 구조를 설계하고 데이터 로더로 미니배치 단위로 학습시키며 학습 곡선을 이용해 학습이 적절하게 수행되었는지 파악합니다. 특히 모델 구조를 설계할 때 손실 함수와 옵티마이저를 지정해야 합니다.

또한 이번 차시에서는 딥러닝의 핵심 데이터 구조인 텐서 개념과 더불어 텐서 데이터 세트와 데이터 로더를 설명했습니다. 그리고 데이터와 모델을 디바이스(cpu 혹은 cuda)에 할당하거나 다른 디바이스로 옮기는 방법도 익혔습니다.

끝으로 가중치 업데이트를 다뤘습니다. 가중치는 딥러닝 모델이 입력 데이터를 학습하는 데 있어 핵심적인 요소입니다. 가중치는 학습 초기에는 랜덤하게 설정되지만, 학습 과정에서 점차 최적의 값으로 조정됩니다. 가중치 조정은 손실 함수를 최소화하는 방향으로 가중치를 업데이트하는 과정이며, 이를 위해 경사 하강법과 같은 최적화 기법을 사용합니다. 가중치 조정이 제대로 이루어지면 모델은 패턴을 정확히 학습하고 새로운 데이터에서도 좋은 성능을 낼 수 있습니다.

이제 딥러닝의 기본 구성과 학습 원리를 익혔다면 본격적으로 모델을 만들 시간입니다. 다음 차시에서는 수치값을 예측하는 회귀 문제를 중심으로 딥러닝 모델을 직접 설계하고 학습시키는 방법을 살펴봅니다. 단계별로 단순한 모델에서 시작해 점차 복잡한 구조의 모델을 학습합니다.